

Experiment 11 — Orchestration using Docker Compose & Docker Swarm

Objective

To understand container orchestration and deploy a multi-container application using Docker Swarm. The experiment demonstrates scaling, load balancing, and self-healing capabilities compared with Docker Compose.

Theory

What is Container Orchestration?

Container orchestration is the **automatic management of containers**.

It handles tasks such as:

- scaling containers
- restarting failed containers
- distributing traffic across containers
- running services across multiple machines

An analogy is a **restaurant manager** who ensures enough staff are working, replaces unavailable staff, and distributes customers efficiently.

Progression of Container Management Tools

docker run → Docker Compose → Docker Swarm → Kubernetes

Tool	Purpose
docker run	run a single container
Docker Compose	run multiple containers together
Docker Swarm	orchestrate containers across nodes
Kubernetes	advanced orchestration platform

This lab focuses on **moving from Compose to Swarm orchestration**.

Prerequisites

- Docker installed
- Docker Swarm enabled

- WordPress + MySQL Compose file from Experiment 6

Docker Compose File

version: '3.9'

services:

db:

image: mysql:5.7

container_name: wordpress_db

restart: always

environment:

MYSQL_ROOT_PASSWORD: rootpass

MYSQL_DATABASE: wordpress

MYSQL_USER: wpuser

MYSQL_PASSWORD: wppass

volumes:

- db_data:/var/lib/mysql

wordpress:

image: wordpress:latest

container_name: wordpress_app

depends_on:

- db

ports:

- "8080:80"

restart: always

environment:

WORDPRESS_DB_HOST: db:3306

WORDPRESS_DB_USER: wpuser

WORDPRESS_DB_PASSWORD: wppass

WORDPRESS_DB_NAME: wordpress

volumes:

- wp_data:/var/www/html

volumes:

db_data:

wp_data:

Task 1 — Stop Existing Containers

First ensure no containers are running.

`docker compose down -v`

`docker ps`

Expected result: no containers running.

Task 2 — Initialize Docker Swarm

Enable Swarm mode:

```
ubuntu@chanteagocci:~$ docker swarm init --advertise-addr 172.22.87.136
Swarm initialized: current node (v6zzfufq8onqnlapf8kloieez) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-lwnml3ttypv0oo7p2yy41blofcke95gv3ex0wh3eo55kagh9e9-6qsd51g19v34ta54kbinuu5p9 172.22.87.136:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

This command:

- enables swarm mode
- makes the current machine the **manager node**
- generates a join token for worker nodes

Verify swarm status:

```
ubuntu@chanteagocci:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
v6zzfufq8onqnlapf8kloieez * LUCIF3R        Ready     Active           Leader             29.4.0
```

Expected output shows the node status as **Ready / Leader**. :contentReference[oaicite:4]{index=4}

Task 3 — Deploy Application as a Stack

Deploy the compose file as a **stack**.

```

Ignoring unsupported options: restart
Ignoring deprecated options:
container_name: Setting the container name is not supported.
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress

```

Explanation:

Command	Meaning
docker stack deploy	deploy application in swarm
-c docker-compose.yml	specify compose file
wpstack	stack name

Behind the scenes Swarm:

1. reads the compose file
2. creates services
3. manages containers automatically.

Task 4 — Verify Deployment

List services:

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
s0shrokwbcl	wpstack_db	replicated	1/1	mysql:5.7	
t7fl7sa6r0t5	wpstack_wordpress	replicated	0/1	wordpress:latest	*:8080->80/tcp

Example output:

```
ID NAME MODE REPLICAS IMAGE
```

```
xxxx wpstack_db replicated 1/1 mysql:5.7
```

```
xxxx wpstack_wordpress replicated 1/1 wordpress:latest
```

Inspect running tasks:

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
s88w74v0994n	wpstack_wordpress.1	wordpress:latest	LUCIF3R	Running	Running 3 seconds ago		

Check containers:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2faf08231f4a	wordpress:latest	"docker-entrypoint.s..."	24 seconds ago	Up 23 seconds	80/tcp	wpstack_wordpress.1.s8
8w74v0994ntwa4m1tulwuv	mysql:5.7	"docker-entrypoint.s..."	About a minute ago	Up About a minute	3306/tcp, 33060/tcp	wpstack_db.1.twogu8ihb
ccefbcf3aad3						
1gpe51wn651vcw7r						

Containers now have names like:

wpstack_wordpress.1.xxxx

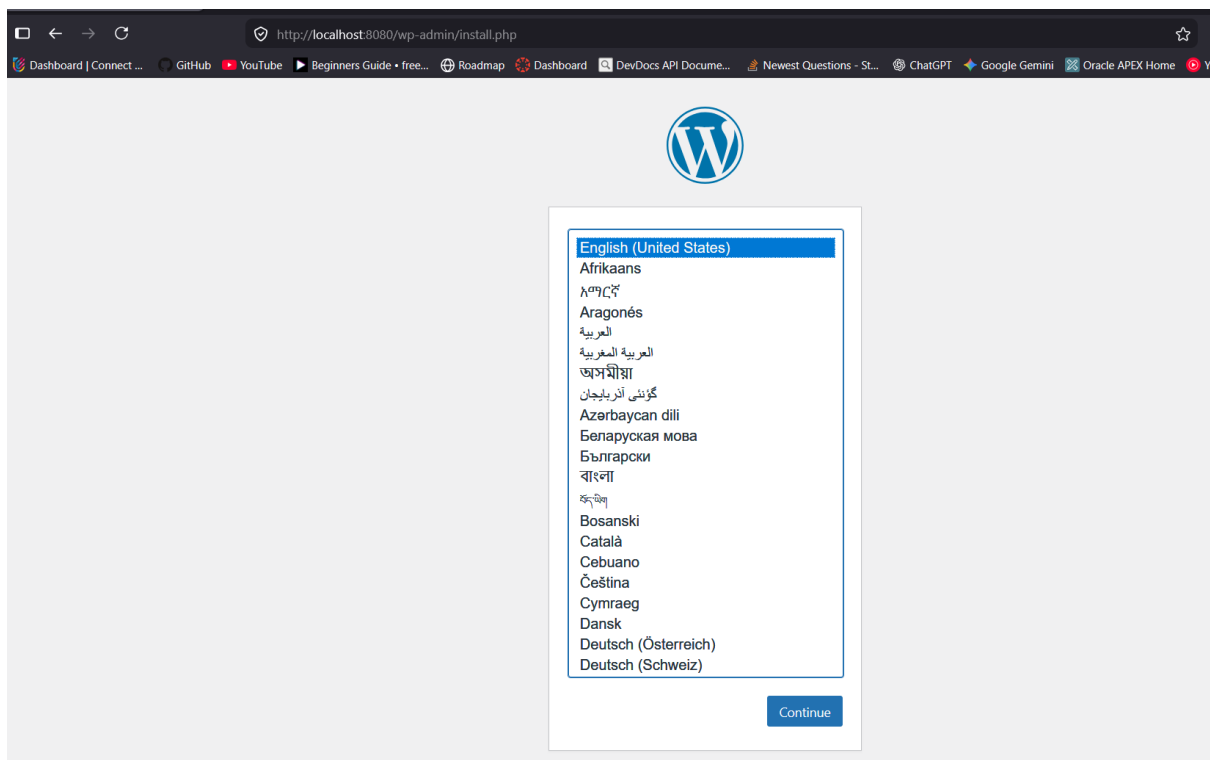
wpstack_db.1.xxxx

Swarm now **manages containers automatically**.

Task 5 — Access the Application

Open browser:

<http://localhost:8080>



You should see the **WordPress setup page**.

Even though Swarm manages the containers, the application behaves the same.

Task 6 — Scale the Application

Scale WordPress service from **1 to 3 replicas**.

```
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running  [=====>]
2/3: running  [=====>]
3/3: running  [=====>]
verify: Service wpstack_wordpress converged
```

Verify scaling:

```
anirudh_chawla@LUCIF3R:~/exp6$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
s0shrokwb1       wpstack_db          replicated          1/1                 mysql:5.7
t7fl7sa6r0t5     wpstack_wordpress   replicated          3/3                 wordpress:latest     *:8080->80/tcp

anirudh_chawla@LUCIF3R:~/exp6$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                NODE                DESIRED STATE       CURRENT STATE       ERROR             PORTS
s88w74v0994n     wpstack_wordpress.1 wordpress:latest     LUCIF3R            Running              Running 7 minutes ago
apw85cdg40or     wpstack_wordpress.2 wordpress:latest     LUCIF3R            Running              Running about a minute ago
p2ct5h4opb1r     wpstack_wordpress.3 wordpress:latest     LUCIF3R            Running              Running about a minute ago

anirudh_chawla@LUCIF3R:~/exp6$ docker ps | grep wordpress
ff5ba9601355     wordpress:latest    "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp              wpstack_wordpress.3.p2
ct5h4opb1rb8ygnv67n0zvf     wordpress:latest    "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp              wpstack_wordpress.2.ap
f8b6d250aad8     wordpress:latest    "docker-entrypoint.s..." About a minute ago   Up About a minute   80/tcp              wpstack_wordpress.2.ap
w85cdg40or1b0b5lrcwepz1     wordpress:latest    "docker-entrypoint.s..." 7 minutes ago       Up 7 minutes        80/tcp              wpstack_wordpress.1.s8
8w74v0994ntwa4mtulwvvt
```

Now **three WordPress** containers run simultaneously.

Load Balancing

Even with multiple containers, users still access:

<http://localhost:8080>

Swarm creates an **internal load balancer** that distributes traffic across all containers automatically.

Task 7 — Test Self-Healing

Swarm automatically replaces failed containers.

Find a WordPress container:

```
docker ps | grep wordpress
```

Kill it:

```
docker kill <container-id>
```

Check service tasks:

```
anirudh_chawla@LUCIF3R:~/exp6$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                NODE                DESIRED STATE       CURRENT STATE       ERROR             PORTS
s88w74v0994n     wpstack_wordpress.1 wordpress:latest     LUCIF3R            Running              Running 11 minutes ago
apw85cdg40or     wpstack_wordpress.2 wordpress:latest     LUCIF3R            Running              Running 5 minutes ago
p2ct5h4opb1r     wpstack_wordpress.3 wordpress:latest     LUCIF3R            Running              Running 5 minutes ago
```

Swarm detects the failure and **creates a replacement container automatically**.

Task 8 — Remove the Stack

Clean up all services:

```
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Verify removal:

```
ID          NAME          MODE          REPLICAS    IMAGE          PORTS
```

Containers should be removed.

Compose vs Swarm

Feature	Docker Compose	Docker Swarm
Scope	single host	multi-node cluster
Scaling	basic	built-in service scaling
Load balancing	no	yes
Self-healing	no	yes
Rolling updates	no	yes
Service discovery	container names	DNS + VIP

Compose is best for development, while Swarm supports production orchestration.

Observations

1. The same Compose YAML file can be reused in Swarm.
2. In Swarm, you manage **services instead of containers**.
3. Swarm resolves port conflicts using an internal load balancer.

Learning Outcome Questions

1. Why is Docker Compose not sufficient for production environments?
2. What does docker stack deploy do differently from docker compose up?

3. How does Swarm provide self-healing?
 4. What happens if a container managed by Swarm is killed?
 5. Can the same Compose file be used for development and production?
-

Conclusion

This experiment demonstrated container orchestration using Docker Swarm.

A multi-container WordPress application was deployed using a Compose file, scaled to multiple replicas, and tested for self-healing capabilities. Swarm provides built-in load balancing, automatic recovery, and cluster management, making it more suitable for production environment